

KUCampus Bite – Risk Response Strategy Table

Risk No.	Risk Category	Risk Name	Risk Response Strategy	Description
1.	Technical	API Integration	Mitigate	Ensure APIs for payment and canteen systems are reliable by using backups, testing endpoints, and adding retry logic.
		Scalability	Mitigate	Build the system with scalable architecture to handle increasing student traffic and canteen orders smoothly.
		Cross-browser compatibility	Mitigate	Test the website on all major browsers and devices to make sure it works properly for every student user.
		Security	Avoid	Use HTTPS, secure login, and database encryption to prevent data leaks or unauthorized access.
		Technical Debt	Mitigate	Regularly review and clean up code, follow standards, and document updates to prevent performance issues later.
2.	Project Management	Unrealistic Deadlines	Avoid	Set realistic timelines with canteen partners and developers to ensure proper testing and stable releases
		Communication Failure	Mitigate	Hold regular meetings between canteen owners, riders, and developers to ensure smooth coordination.
3.	Organizational	Canteen Staff Cooperation	Accept	Manage through regular communication, training, and positive motivation to ensure canteen owners cooperate with the system.
		User Resistance	Mitigate	Conduct awareness sessions and demos to help students and staff understand how to use the KU Campus Bite website.

4.	External	Network Outage	Transfer	Use a reliable internet provider and maintain backup connections to minimize downtime for orders and updates.
		Competitor Launch	Accept	Monitor competitors and focus on offering better service and features to maintain KU Campus Bite's user loyalty.
		Service Failure	Transfer	Host the system with a reliable provider and use SLAs (Service Level Agreements) to ensure uptime and quick recovery.
5.	Requirement/ Scope	Unclear Requirements	Mitigation	Clarify all system requirements early by conducting meetings, preparing an SRS document, and confirming features with stakeholders before development.
		Scope Creep	Avoid	Prevent unapproved feature additions by defining clear project boundaries, maintaining a scope document, and enforcing a strict change control process.
		Student Feedback	Mitigation	Collect structured feedback only after major milestones, prioritize feasible suggestions, and document others for future updates to prevent development delays.
6.	Resources	Team Member Absence	Mitigation	Reduce dependency on individuals by cross-training team members, maintaining shared documentation, and reassigning tasks when a member is unavailable.
		Lack of Expertise	Mitigation	Enhance team skills through peer mentoring, online tutorials, and supervisor guidance to ensure all modules (backend, frontend, database) are completed efficiently.
		Financial Constraints	Avoid	Minimize costs by using free-tier hosting services, open-source tools, and budgeting resources carefully to stay within financial limits.
		Limited Time	Mitigation	Meet deadlines by dividing the project into milestones, prioritizing essential features first, and using project management tools like Trello for progress tracking.
7.	Quality	Buggy Code	Mitigation	Implement testing at each stage, review code regularly, and use a bug-tracking

				system to find and fix errors early.
		Poor UI	Mitigation	Get feedback from users, use a clean and simple design, and test to ensure the interface is easy to use.
		Inaccurate Data	Mitigation	Use input checks and auto price calculations, test orders regularly, and keep the database updated in real time.